CS402 – High-Performance Computing

Introduction

Gihan Mudalige
Professor
Department of Computer Science
University of Warwick
g.mudalige@warwick.ac.uk

A Third Pillar of Science ?

SC2022 – Scientific Visualization and Data Analytics Showcase

The Scientific Method and the Two Pillars of Science

Theory

1. Make and observation
2. Ask a question
3. Form a hypothesis – an explanation
4. Make a prediction based on the hypothesis

Experiment

5. Test the prediction
6. Iterate: use the results to refine hypothesis or make a new one or new predictions

Theorists make models of some phenomenon / real world

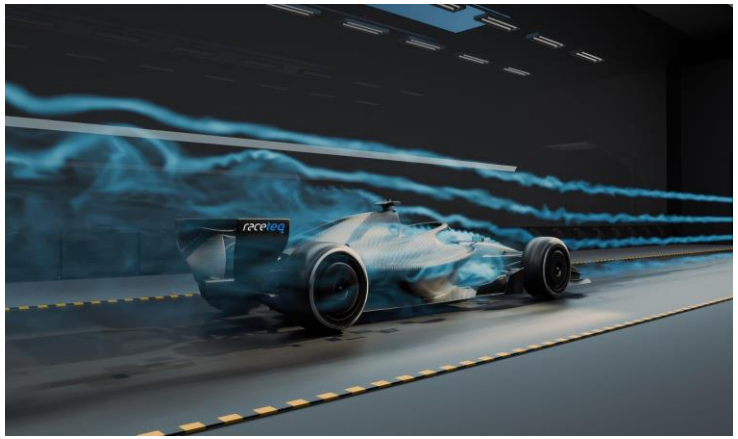
Experimentalists tests the predictions from theory

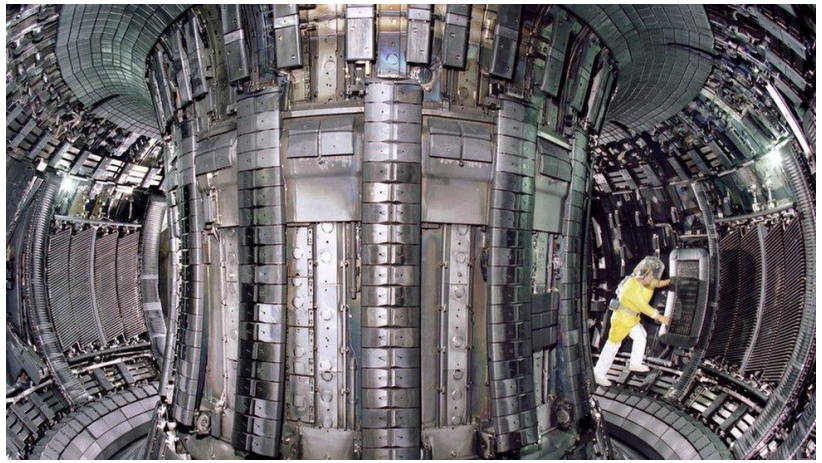
The two classic pillars underpinning scientific method

If experiments yield different results / different effects not supported by theory – need new theory!

Third Pillar of Science ?

- Some experiments are
  - economically infeasible - Designing aero-engines need very expensive wind tunnel experiments
  - ethically inappropriate - Designing a new radiation treatment cannot be tested on patients
  - impossible to construct - New theory of the Big Bang likely never possible to construct small experiment





Third Pillar of Science ?

- Some equations cannot be solved analytically – i.e. find an exact, closed-form expression for the solution.
  - Velocity and acceleration of a free object in space

$$v(t) = \frac{du}{dt}, \quad a(t) = \frac{d^2u}{dt^2} \quad , \quad F = ma = m\frac{d^2u}{dt^2}$$

  - Gravitational force

$$F = -\frac{GMm}{r^2} \quad , \quad \frac{d^2u}{dt^2} = \frac{F}{m}$$

  - One object orbiting a much larger object (e.g. planet around Sun) :

$$\frac{d^2u}{dt^2} = -\frac{GM}{r^2}$$

  - Three objects all attracting each other :

$$m_i\frac{d^2\mathbf{r}_i}{dt^2} = \sum_{j\neq i} G\frac{m_i m_j}{|\mathbf{r}_i - \mathbf{r}_j|^3}(\mathbf{r}_j - \mathbf{r}_i), \quad i = 1, 2, 3$$

Analytical solution for object falling under constant gravity

$$\frac{d^2u}{dt^2} = -g$$

Integrate twice :

$$\frac{du}{dt} = -gt + v_0$$

$$u(t) = -\tfrac{1}{2}gt^2 + v_0 t + u_0$$

Gives the analytical solution – a formula for position as a function of time

Need to solve numerically – approximate the solution,
refining estimates iteratively until a desired level of precision is achieved.

Third Pillar of Science ?

❑ Single Pendulum

- Equation of motion (nonlinear)

$$\theta''(t) + \frac{g}{L}\sin\theta(t) = 0$$

- Small-angle ($\theta \ll 1, \sin\theta \approx 0$) gives a linear ODE

$$\theta'' + \omega_0^2\theta = 0, \quad \omega_0 = \sqrt{\frac{g}{L}}$$

- Solution – with initial angle θ_0, initial angular velocity $\omega_0^{(0)}$

$$\theta(t) = \theta_0 \cos\omega_0 t + \frac{\omega_0^{(0)}}{\omega_0}\sin\omega_0 t$$

MIT Physics Instructional Resources Lab
(https://www.youtube.com/watch?v=czLIj-4suOk)

❑ Double Pendulum … !!

$$\theta_1'' = \frac{-g\,(2\,m_1 + m_2)\sin\theta_1 - m_2\,g\sin(\theta_1 - 2\,\theta_2) - 2\sin(\theta_1 - \theta_2)\,m_2\,(\theta_2'^2\,L_2 + \theta_1'^2\,L_1\cos(\theta_1 - \theta_2))}{L_1\,(2\,m_1 + m_2 - m_2\cos(2\,\theta_1 - 2\,\theta_2))}$$

$$\theta_2'' = \frac{2\sin(\theta_1 - \theta_2)\,(\theta_1'^2\,L_1\,(m_1 + m_2) + g(m_1 + m_2)\cos\theta_1 + \theta_2'^2\,L_2\,m_2\cos(\theta_1 - \theta_2))}{L_2\,(2\,m_1 + m_2 - m_2\cos(2\,\theta_1 - 2\,\theta_2))}$$

- Coupled, nonlinear ODEs – no general closed-form solution, motion typically chaotic
- Solve numerically – e.g. Runge-Kutta 4

Third Pillar of Science : Computer Simulation (?)

- Real-World Systems are Too Complex for Analytic Solutions
  - **Weather & Climate:** turbulent, multiscale flows → Navier–Stokes + thermodynamics
  - Navier-Stokes equation for incompressible fluid flow:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f},$$

$$\nabla \cdot \mathbf{u} = 0$$

Where u = velocity, p = pressure, v = viscosity
Coupled with thermodynamics + moisture transport.

  - **Aerodynamics:** aircraft design → nonlinear PDEs, 3D geometries
  - Compressible Navier-Stokes (Euler + viscous terms)

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0,$$

$$\frac{\partial (\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u} + pI) = \nabla \cdot \tau,$$

$$\frac{\partial E}{\partial t} + \nabla \cdot \big((E + p)\mathbf{u}\big) = \nabla \cdot (k \nabla T) + \Phi$$

Describes density ρ, momentum and energy transport in air

Third Pillar of Science : Computer Simulation (?)

- Real-World Systems are Too Complex for Analytic Solutions
  - **Finance:** option pricing → stochastic PDEs, high-dimensional
  - Black-Scholes PDE for options pricing

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0$$

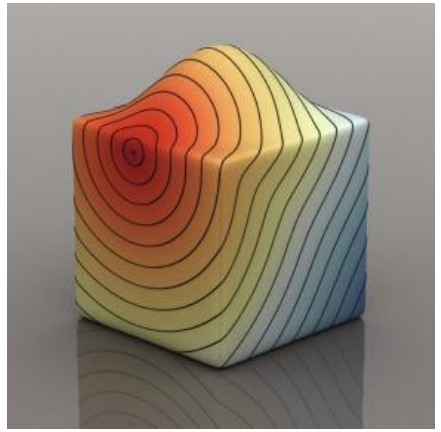
where $V(S,t)$ = option price, σ = volatility, r = interest rate
in multi-asset case, becomes a high dimensional PDE.

  - **Physics/Materials:** quantum systems, plasma, nuclear interactions
  - E.g. Time-dependent Schrödinger equation (quantum mechanics)

$$i\hbar\frac{\partial \psi}{\partial t} = -\frac{\hbar^2}{2m}\nabla^2\psi + V(x,y,z)\psi$$

where ψ = wave function, V = potential

- Example – Heat Diffusion: Models any process of diffusion (spread/averaging) in time, such as heat
  - **Thermal Applications:** Heat transfer in materials, building design, electronics cooling, furnaces
  - **Climate science**: diffusion of heat in oceans and atmosphere, pollutant dispersion in air or water
  - **Automotive/Aerospace:** Heat diffusion in turbine blades, brake discs, rocket nozzles



https://www.sciencedirect.com/topics/computer-science/heat-diffusion

- Mathematical model: 1D PDE

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

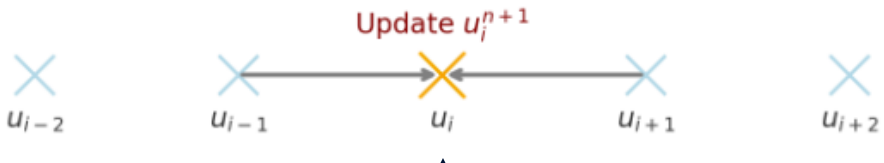
$u(x,t)$ - temperature (or concentration)

α - diffusion constant

$x \in [0, L], t > 0$ and boundary conditions : e.g. $u(0,t) = u(L,t) = 0$

  - Discretization (Finite Differences)

    Replace continuous space and time with a grid and discretize



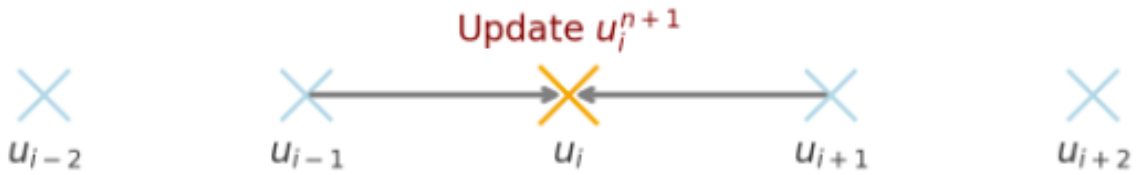
$$u(x,t) \rightarrow u_i^n \quad \text{with } i = \text{space index}, \; n = \text{time index}$$

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{\Delta x^2}$$

$$u_i^{n+1} = u_i^n + \frac{\alpha \Delta t}{\Delta x^2}\left(u_{i-1}^n - 2u_i^n + u_{i+1}^n\right)$$

Stencil computation : each grid point depends on its neighbours *3-point Laplacian*

Modern Numerical Solvers needs High-Performance Computing



- 1D Solver code (in Python !)

```
for n in range(N_timesteps):
    for i in range(1, N_points-1):
        u_new[i] = u[i] + alpha*dt/dx**2 * (u[i-1] - 2*u[i] + u[i+1])
    u[:] = u_new[:]
```

- Each loop iteration corresponds to updating one grid point
- Complexity:

$$O(N_{\text{points}} \times N_{\text{timesteps}})$$

- Significantly more complex for 3D:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \quad \text{in } (x, y, z) \in \Omega,\ t > 0$$

Boundary conditions : Dirichlet boundary condition – fix value of the solution u on the boundary of the domain $\partial\Omega$

$$u(x, y, z, t) = 0 \quad \text{for } (x, y, z) \in \partial\Omega,\ t > 0$$

The boundary is held at a constant temperature (0 in this case)

- Discretization:

(1) Lay down a uniform grid in space and time

spatial nodes : $x_i = i\,\Delta x$ for $i = 0,1,2,\ldots,N_x$ and $y_j = j\,\Delta y$, and $z_k = k\,\Delta z$

time levels : $t^n = n\,\Delta t, n = 0,1,2,\ldots$

then let $u^n_{i,j,k} \approx u(x_i, y_j, z_k, t^n)$

(2) Approximate the time derivative (forward Euler)

$$\left.\frac{\partial u}{\partial t}\right|^n_{i,j,k} \approx \frac{u^{n+1}_{i,j,k} - u^n_{i,j,k}}{\Delta t}$$

(3) Second spatial derivatives (central differences):

$$\left.\frac{\partial^2 u}{\partial x^2}\right|^n_{i,j,k} \approx \frac{u^n_{i+1,j,k} - 2u^n_{i,j,k} + u^n_{i-1,j,k}}{\Delta x^2},$$

$$\left.\frac{\partial^2 u}{\partial y^2}\right|^n_{i,j,k} \approx \frac{u^n_{i,j+1,k} - 2u^n_{i,j,k} + u^n_{i,j-1,k}}{\Delta y^2},$$

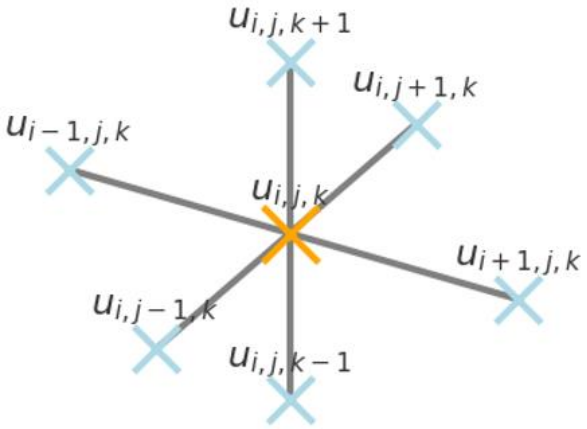
$$\left.\frac{\partial^2 u}{\partial z^2}\right|^n_{i,j,k} \approx \frac{u^n_{i,j,k+1} - 2u^n_{i,j,k} + u^n_{i,j,k-1}}{\Delta z^2}.$$

$$\begin{aligned} u^{n+1}_{i,j,k} = u^n_{i,j,k} &+ r_x\left(u^n_{i+1,j,k} - 2u^n_{i,j,k} + u^n_{i-1,j,k}\right) \\ &+ r_y\left(u^n_{i,j+1,k} - 2u^n_{i,j,k} + u^n_{i,j-1,k}\right) \\ &+ r_z\left(u^n_{i,j,k+1} - 2u^n_{i,j,k} + u^n_{i,j,k-1}\right), \end{aligned} \qquad r_x = \alpha\frac{\Delta t}{\Delta x^2}, \qquad r_y = \alpha\frac{\Delta t}{\Delta y^2}, \qquad r_z = \alpha\frac{\Delta t}{\Delta z^2}$$

3D finite difference stencil : 7-point Laplacian

Modern Numerical Solvers needs High-Performance Computing

$$
\begin{aligned}
u_{i,j,k}^{n+1} = u_{i,j,k}^{n} &+ r_x\left(u_{i+1,j,k}^{n} - 2u_{i,j,k}^{n} + u_{i-1,j,k}^{n}\right) \\
&+ r_y\left(u_{i,j+1,k}^{n} - 2u_{i,j,k}^{n} + u_{i,j-1,k}^{n}\right) \\
&+ r_z\left(u_{i,j,k+1}^{n} - 2u_{i,j,k}^{n} + u_{i,j,k-1}^{n}\right),
\end{aligned}
$$

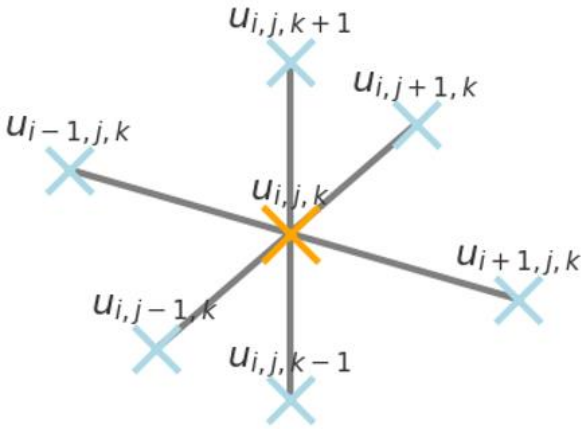


- Real world problems are 3D
  - Number of unknowns = $N_x \times N_y \times N_z$
  - Example weather model with 2000^3 grid points = 8 Billion unknowns
  - At each time-step, update billions of floating-point operations
  - Thousand time-steps → over trillions of operations !

- Computational Resources
  - **Memory:** Billion unknowns $8\times(2000^3)$ bytes ≈ 64 GB just for u, multiple fields (e.g. u_{new}) and more.
  - **Compute cost**: Each timestep updates every cell → $O(N_x \times N_y \times N_z)$
    Thousands of timesteps ⇒ **trillions of FLOPs**.
  - **Communication:** Nearest neighbour communication every time-step, high bandwidth, low-latency
  - **Time-to-solution** : cannot compute one point after another sequentially

Modern Numerical Solvers needs High-Performance Computing

$$
\begin{aligned}
u^{n+1}_{i,j,k} = u^n_{i,j,k} &+ r_x\left(u^n_{i+1,j,k} - 2u^n_{i,j,k} + u^n_{i-1,j,k}\right) \\
&+ r_y\left(u^n_{i,j+1,k} - 2u^n_{i,j,k} + u^n_{i,j-1,k}\right) \\
&+ r_z\left(u^n_{i,j,k+1} - 2u^n_{i,j,k} + u^n_{i,j,k-1}\right),
\end{aligned}
$$



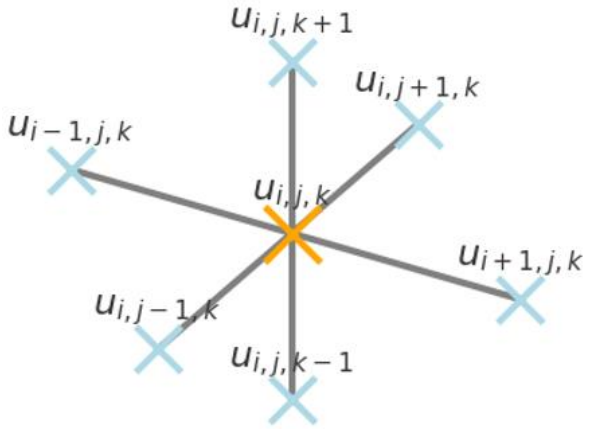
- Time to solution
  - FLOP cost per per grid point : 9 adds and 6 mults = 15 FLOPs
  - Per time step FLOPs: $2000^3 \times 15 = 8 \times 10^9 \times 15 \approx 120$ GFLOPs
  - 4 GHz CPU (≈1 flop/cycle): 4 GFLOP/s → 120/4 ≈ 30 sec/step

  - **SIMD/Vector units** (SSE, AVX, AVX-512) : apply one instruction to 4 - 8 doubles at once
  - **FMA instructions**: do a multiply and an add in a single operation

  - With AVX2 on doubles: 256-bit registers hold 4 doubles, 1x FMA instruction = 8FLOPs (4 mults + 4 adds)
  - CPU cores can issue 2 FMAs per cycle → 16 FLOPs/cycle. (e.g. AMD Zen 2/3/4, Intel Sapphire Rapids)
  - But assuming an 8 FLOPs/cycle conservative estimate one core will give : 4GHz × 8 FLOP/cycle = 32 GFLOP/s → 120/32 ≈ <u>4 sec/step</u>

Modern Numerical Solvers needs High-Performance Computing

$$
\begin{aligned}
u^{n+1}_{i,j,k} = u^{n}_{i,j,k} &+ r_x\left(u^{n}_{i+1,j,k} - 2u^{n}_{i,j,k} + u^{n}_{i-1,j,k}\right) \\
&+ r_y\left(u^{n}_{i,j+1,k} - 2u^{n}_{i,j,k} + u^{n}_{i,j-1,k}\right) \\
&+ r_z\left(u^{n}_{i,j,k+1} - 2u^{n}_{i,j,k} + u^{n}_{i,j,k-1}\right),
\end{aligned}
$$

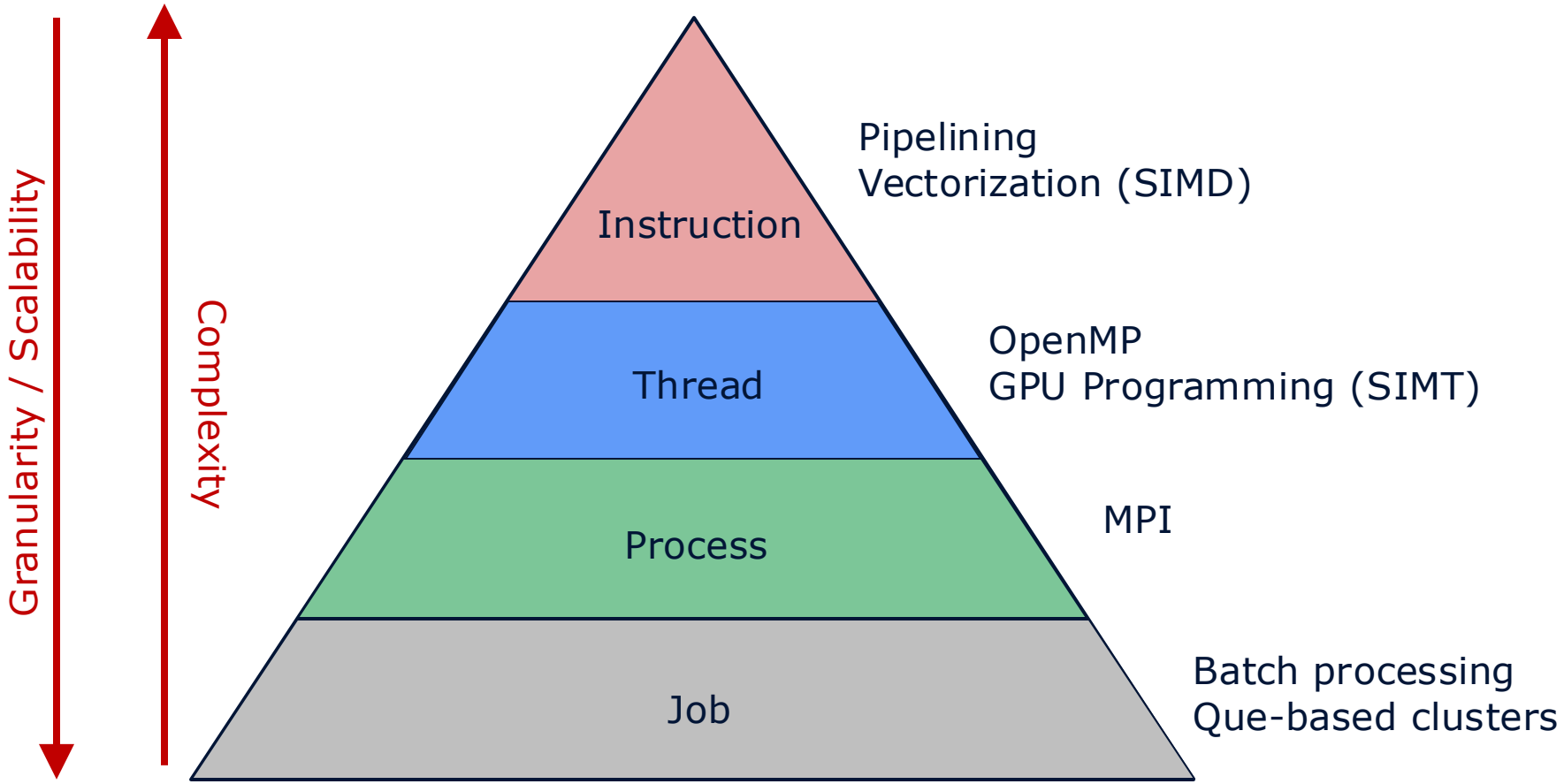


- But time per time step is bandwidth bound
  - Each update touches (roughly) 7 reads + 1 write = 8 doubles ≈ 64 bytes / update (worst case)
  - With good caching (neighbours get reused) : 1 reads + 1 write = 2 doubles ≈ 16 bytes / update (best case)
  - Per time step traffic = 2 to 8 × 10^9 × 64 bytes ≈ 128 GB to 512 GB

  - However on a single CPU sustainable single-thread memory bandwidth is often 20GB/s to 50GB/s (per socket BW is higher, but single core typically will not saturate it)
  - At 20GB/s → 128/20 to 512/20 ≈ <u>6.4 sec/step to 25.6 sec/step</u>
  - At 50GB/s → 128/50 to 512/50 ≈ <u>2.56 sec/step to 10.2 sec/step</u>
  - Therefore <u>time to solution</u> for 1000 time steps = **0.71 to 7.1 hours !**
- Big caveat : The RAM requirement (~128 GB for at least both u and u_{new}) will mean that a single workstation will not fit the problem

Requires large-scale parallel computing, supercomputers / HPC to solve within a tractable/realistic time

Levels of Parallelism



- Simulation is the only practical route for solving many modern scientific/engineering questions
  - The governing equations are clear, but they are too complex to solve exactly
  - Only numerical approximation combined with HPC lets us tackle problems at real-world scale.

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0,$$

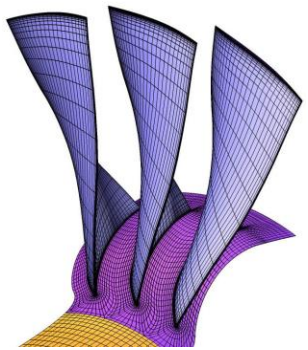
$$\frac{\partial (\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u} + pI) = \nabla \cdot \tau,$$

$$\frac{\partial E}{\partial t} + \nabla \cdot \big((E + p)\mathbf{u}\big) = \nabla \cdot (k \nabla T) + \Phi$$

```
while ( error > tol && iter < iter_max ) {
  error = 0.0;
  for( int j = 1; j < jmax+1; j++ ) {
    for( int i = 1; i < imax+1; i++) {
      Anew[(j)*(imax+2)+i] = 0.25f * ( A[(j)*(imax+2)+i+1] + A[(j)*(imax+2)+i-1]
          + A[(j-1)*(imax+2)+i] + A[(j+1)*(imax+2)+i]);
      error = fmax( error, fabs((float)(Anew[(j)*(imax+2)+i]-A[(j)*(imax+2)+i])));
    }
  }

  for( int j = 1; j < jmax+1; j++ ){
    for( int i = 1; i < imax+1; i++) {
      A[(j)*(imax+2)+i] = Anew[(j)*(imax+2)+i];
    }
  }
  if(iter % 10 == 0) printf("%5d, %0.6f\n", iter, (float)error);
  iter++;
}
```

1. Mathematical model:
PDEs, stochastic equations



2. Numerical Approximation: Discretization → algebraic systems (stencil updates, matrices)

3. Computer Simulation code (Parallel C/C++/Fortran, OpenMP, CUDA, MPI)

4. HPC
(Execute on clusters of CPUs/GPUs, FPGAs)

Diverse (Parallel) Hardware Landscape

- Traditional CPUs
  - Intel, AMD, ARM, IBM
  - multi-core (> 20 currently)
  - Deep memory hierarchy (cache levels and RAM)
  - longer vector units (e.g. AVX-512)
- GPUs
  - NVIDIA (H100), AMD (MI300) , Intel (Xe GPUs)
  - Many-core (> 1024 simpler SIMT cores)
  - CUDA cores, Tensor cores
  - Cache, Shared memory, HBM (3D stacked DRAM)
  - Unified memory e.g. AMD Instinct MI300
- Heterogeneous Processors
  - Different core architectures over the past years
  - ARM *big.LITTLE*
  - NVIDIA *Grace.Hopper*
- XeonPhi (discontinued)
  - Many-core – based on simpler x86 cores
  - MCDRAM (3D stacked DRAM)
- FPGAs
  - Xilinx (AMD) and Intel/Altera
  - Various configurations
  - Low-level language / HLS tools
  - Significant energy savings
- AI-specialized hardware
  - NVIDIA's Hopper/Blackwell Tensor Cores
  - Cerebras Wafer-Scale Engines
  - Google TPUs v5e
  - Graphcore IPUs
  - custom ASICs driven by AI
    [... in the cloud.]
- DSP Processors
  - Matrix 2000+ (MTP) DSP accelerator
- Domain specific Hardware
- Quantum [?]

EXASCALE SYSTEMS

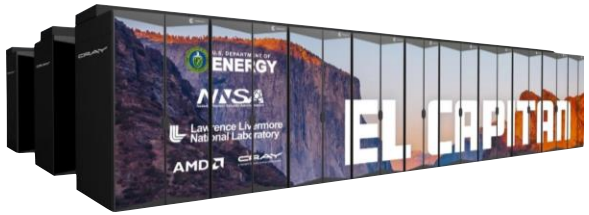
- Exaflop = **10^{18}** double-precision Floating-point calculations

- Current top systems (see https://www.top500.org/)

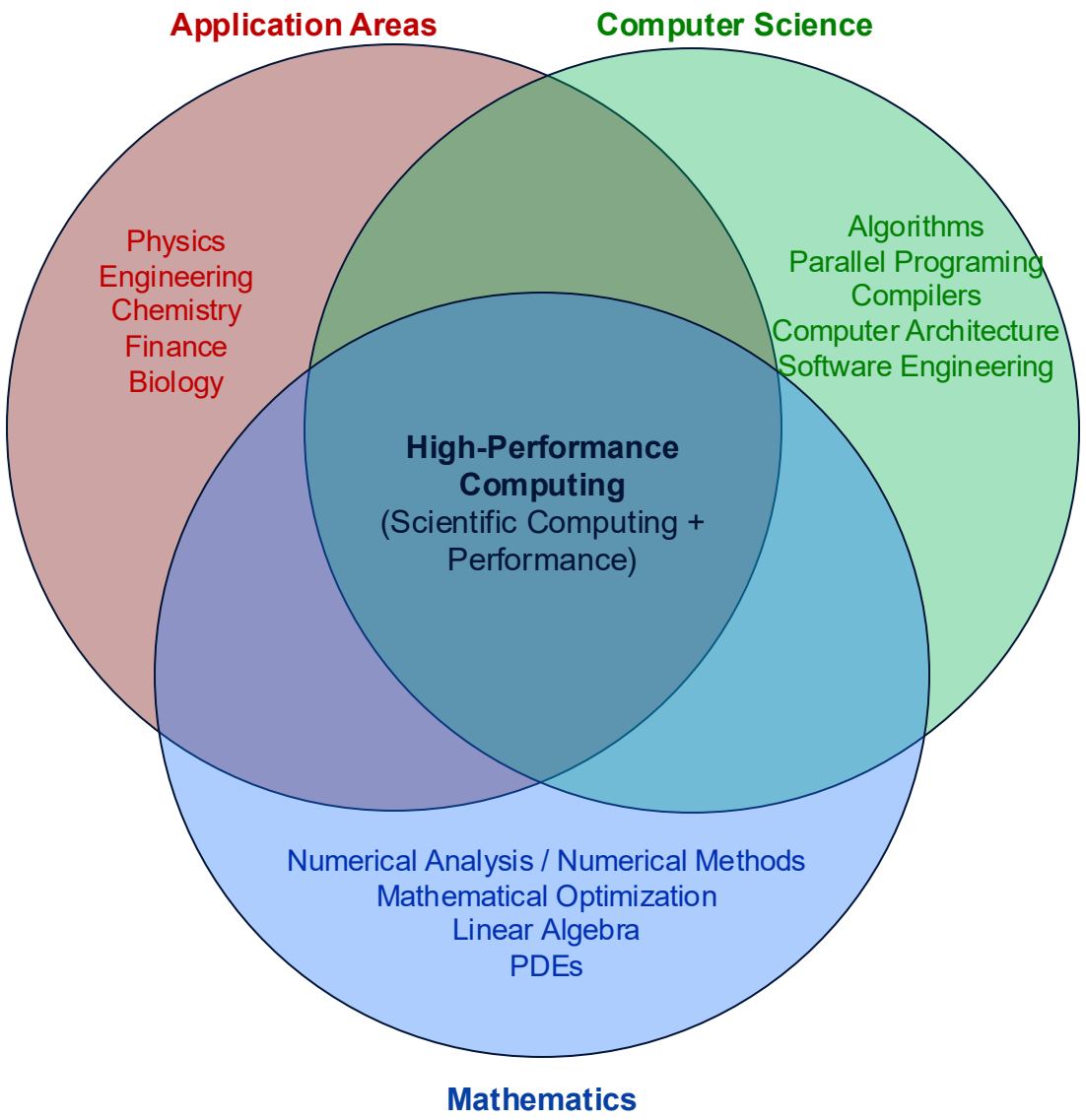
  - Aurora - 1 EFLOP Intel Xeon CPU (Sapphire Rapids with HBM) with Intel Xe GPUs ("Ponte Vecchio")
  - Frontier - 1.3 EFLOP/s AMD EPYC CPUs (Milan) with AMD Instinct GPUs
  - El Capitan – 1.7 EFLOP/s AMD EPYC CPUs (Genoa) with AMD Instinct GPUs

  - LUMI - 0.37 EFLOP/s AMD EPYC CPUs with AMD Instinct GPUs
  - LEONARDO - 0.24 EFLOP/s - Intel Xeon CPUs (Sapphire Rapids) with NVIDIA A100 GPUs
  - Jupiter (Booster) – 0.48 EFLOP/s GH200 Grace Hopper Superchips

  - ARCHER2- 28 PFLOP/s AMD EPYC CPUs (Rome)
  - Isambard-AI - 0.2 EFLOP/s NVIDIA GH200 Superchip
  - [UK Exascale system to be installed in summer 2027]



HPC is Interdisciplinary

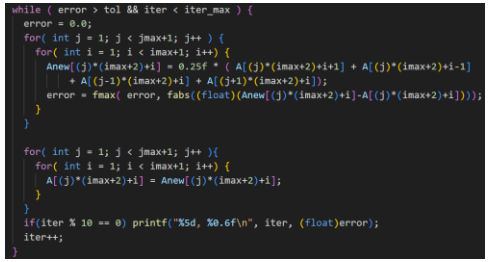


Mainly this module will cover

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0,$$

$$\frac{\partial (\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u} + pI) = \nabla \cdot \tau,$$

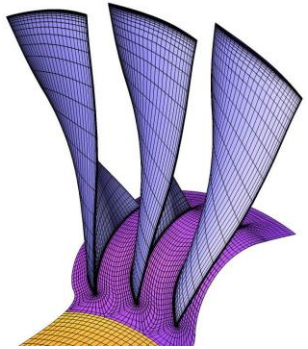
$$\frac{\partial E}{\partial t} + \nabla \cdot \big((E + p)\mathbf{u}\big) = \nabla \cdot (k \nabla T) + \Phi$$



```
while ( error > tol && iter < iter_max ) {
  error = 0.0;
  for( int j = 1; j < jmax+1; j++ ) {
    for( int i = 1; i < imax+1; i++) {
      Anew[(j)*(imax+2)+i] = 0.25f * ( A[(j)*(imax+2)+i+1] + A[(j)*(imax+2)+i-1]
          + A[(j-1)*(imax+2)+i] + A[(j+1)*(imax+2)+i]);
      error = fmax( error, fabs((float)(Anew[(j)*(imax+2)+i]-A[(j)*(imax+2)+i])));
    }
  }

  for( int j = 1; j < jmax+1; j++ ){
    for( int i = 1; i < imax+1; i++) {
      A[(j)*(imax+2)+i] = Anew[(j)*(imax+2)+i];
    }
  }
  if(iter % 10 == 0) printf("%5d, %0.6f\n", iter, (float)error);
  iter++;
}
```

1. Mathematical model:
PDEs, stochastic equations



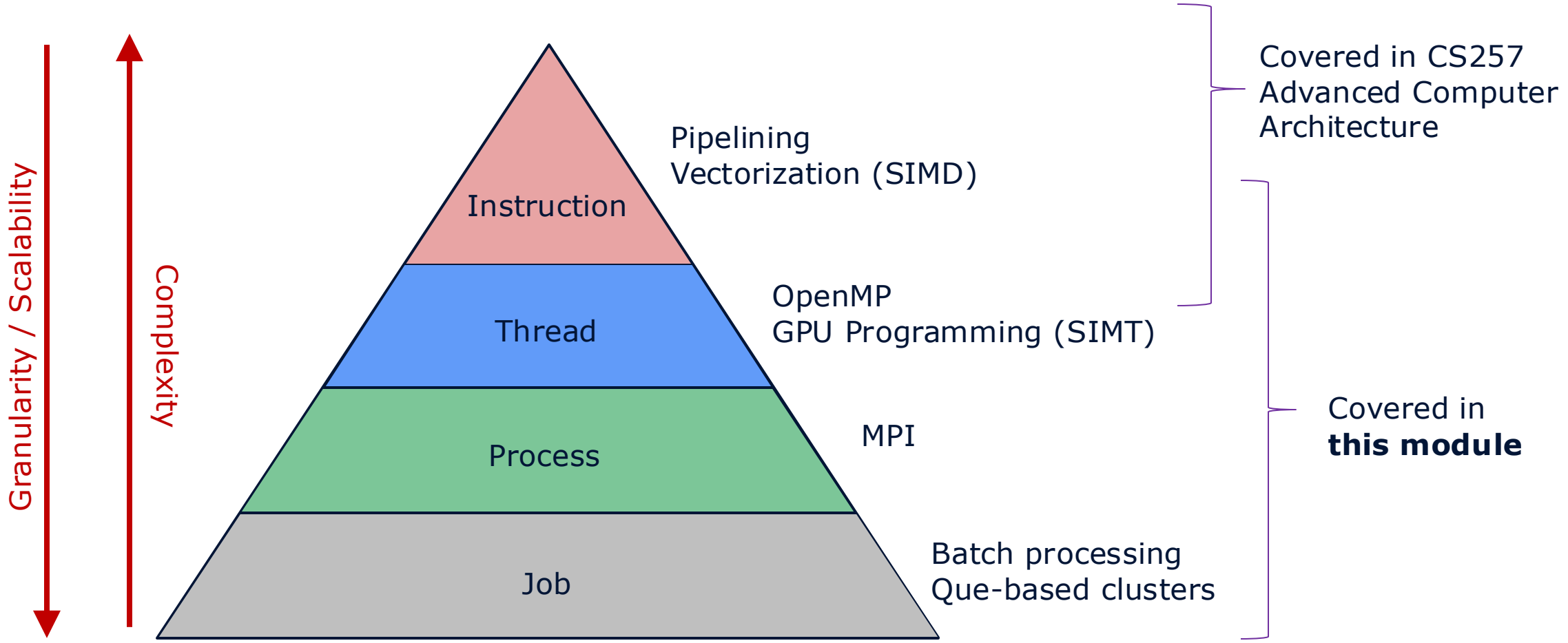
2. Numerical Approximation: Discretization → algebraic systems (stencil updates, matrices)

3. Computer Simulation code (Parallel C/C++/Fortran, OpenMP, CUDA, MPI)

4. HPC
(Execute on clusters of CPUs/GPUs)

Will includes some background from here

Levels of Parallelism



Module Organization

- Prerequisites
  - CS132 - Computer Organisation and Architecture
  - [Optional] CS257 – Advanced Computer Architecture
  - Good programming skills / A motivation to program (and debug) !

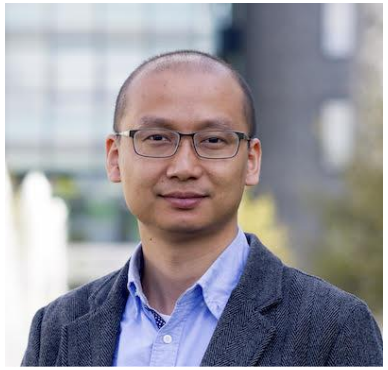
- 20x one-hour lectures + 10x one-hour labs
  - Lectures: Mondays **@15.00-16.00** in FAB0.03 and Tuesday **@13.00-14.00** in L5
  - Labs: Mondays **@16.00-17.00** in MB3.17 or **@17.00-18.00** in MB3.17 (attend according to group)

- Assessment
  - Coursework (30%)
  - Two-hour examination (70%) – during summer exams

MODULE ORGANIZATION

- Lecturers : **Prof. Gihan Mudalige** and **Dr. Ligang He**

g.mudalige@warwick.ac.uk
Room 229
Mathematical Sciences Building



ligang.he@warwick.ac.uk
CS2.05
Computer Science Building

- Teaching Assistants
  - Weitong Liao (Weitong.Liao@warwick.ac.uk)
  - Stephen Xu (Stephen.Xu@warwick.ac.uk)
  - Xuan Huang (Xuan.Huang@warwick.ac.uk)
  - Prakanth Thilakaraj (Prakanth.Thilakaraj@warwick.ac.uk)

- Module Webpage (online material) : https://warwick.ac.uk/fac/sci/dcs/teaching/material/cs402/
- Lecture Capture: Will be made available after each lecture

CS402 Module Content and Weekly Plan

We are here

Deadline for Coursework 1

Deadline for Coursework 2

Week	Lecture Topics	Lecturer	Week	Lab Topics
1	Motivation and context Fundamentals - FP numbers, precision and errors	GM GM	1	C/C++ programming, Introduction to Coursework 1
2	Fundamentals – Parallelism Laws Models of Parallel programming	GM GM	2	OpenMP
3	Shared Memory Parallelism with OpenMP (1) (2)	GM GM	3	Further OpenMP
4	Shared Memory Parallelism with OpenMP (3) Intro to Message Passing Interface	GM LH	4	Coursework 1 Troubleshooting
5	MPI and Types, Advanced MPI	LH LH	5	Introduction to MPI
6	Clusters and Networking GPU (SIMT) Programming (1): CUDA	LH LH	6	Performance Modelling and Advanced MPI
7	GPU (SIMT) Programming (2): CUDA Performance Engineering (1)	LH LH	7	GPUs
8	Performance Engineering (2), (3)	LH LH	8	Troubleshooting Coursework 2
9	Parallel I/O and Visualization Building and Deploying Parallel Applications	LH GM	9	More Troubleshooting Coursework 2
10	Advanced Topics and Latest Developments (1) (2)	GM GM	10	

COURSEWORK

- Computational Fluid Dynamics problem - von Kármán vortex street

  - repeating pattern of swirling vortices, caused by a process known as vortex shedding
  - unsteady separation of flow of a fluid around blunt bodies
  - Named after Hungarian mathematician Theodore von Kármán (1881-1963)

- Coursework code called Karman calculates velocity and pressure of a 2D flow
  - Serial code written in C
  - Structured-mesh decomposition
  - Output written as a binary file
- Objective of the coursework is to parallelize the code using :
  - OpenMP – Run on a multi-core CPU
  - CUDA – Run on an NVIDIA GPU
  - MPI – Run on a Cluster of CPUs (and GPUs !)

TEXT BOOKS

- Tobias Weinzierl. *Principles of Parallel Scientific Computing, A First Guide to Numerical Concepts and Programming Methods*, Springer (2021). https://link.springer.com/book/10.1007/978-3-030-76194-3
  [Free access via Warwick authentication]

- Peter Pacheco and Matthew Malensek *An Introduction to Parallel Programming*, Morgan Kaufmann, 2nd ed, (2021). https://www.cs.usfca.edu/~peter/ipp2/index.html [1st Edition in library QQA76.642.P29]

- Tom Deakin and Timothy G. Mattson, *Programming Your GPU with OpenMP: Performance Portability for GPUs* (2023). https://ompgpu.com/

- Michael T. Heath, Scientific Computing: An Introductory Survey, McGraw-Hill, Revised 2nd ed.(2002) https://heath.cs.illinois.edu/scicomp/

Online Resources

- Parallel Programming and HPC
  - **Argonne National Labs HPC Talks :** https://www.youtube.com/channel/UCfwgjtIQB3puojz_N9ly_Ag
    (Search for :Numerical algorithms and software for extreme Scale, Programming models and languages, Hardware architectures, Software and productivity, Performance tolls and debuggers)
  - **ARCHER2 / EPCC Training Videos :** https://www.youtube.com/channel/UCZi-oBdxoDV5CPEQnhmrCAg
  - **OpenMP Programming : LLNL Tutorial** https://hpc-tutorials.llnl.gov/openmp/
  - **Oxford CUDA Programming Course:** https://people.maths.ox.ac.uk/gilesm/cuda/

  - **Top500 HPC systems:** https://www.top500.org/

- Compilers
  - **Compiler Design (CS325)** : https://warwick.ac.uk/fac/sci/dcs/teaching/material/cs325/
  - **Tobias Grosser MLIR / Compiler talk :** https://www.youtube.com/watch?v=fZSXJzDMEc8&t=834s
  - **Compiler Optimizations :** https://warwick.ac.uk/fac/sci/dcs/teaching/material/cs325/bacon94compiler.pdf

- Application areas
  - 7 Dwarfs - https://www2.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-183.html

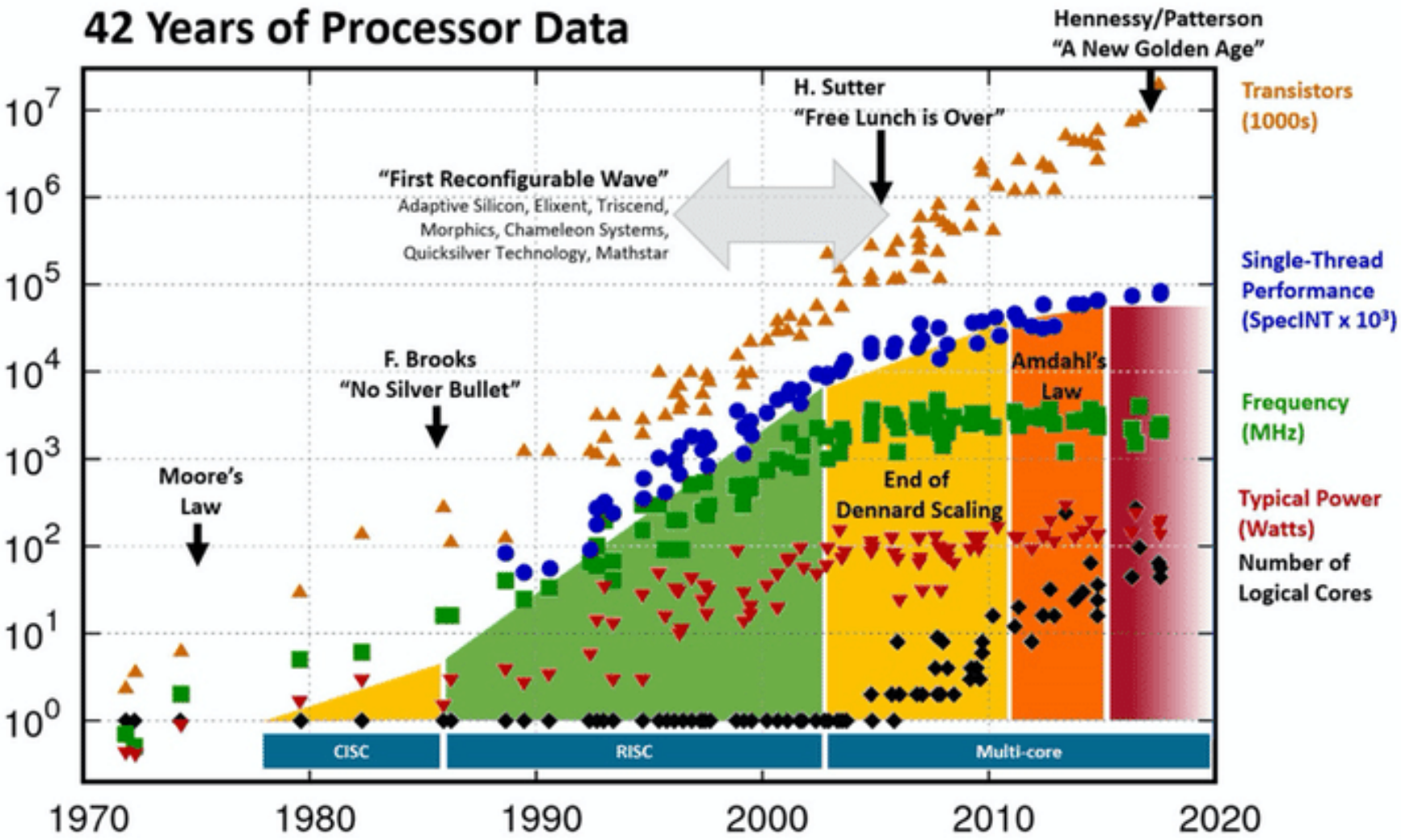
WHY STUDY HPC?

HPC is more than faster computers: it's about enabling discoveries and driving innovation

- **Solve problems that matter** - Climate change, large-scale engineering, renewable energy, astrophysics
- **Work at scale -** Learn how to design algorithms and software that run on thousands of cores or GPUs
- **Broad skills set** – Numerical methods, Compilers, Computer Architecture, Parallel programming, …
- **Cutting-edge of Technology** - Exascale computing, AI accelerators, quantum-HPC hybrids

- Careers
  - **Scientific Research:** computational physics, CFD, climate modelling
  - **Engineering & Industry**: aerospace simulations, automotive design, energy sector
  - **Technology**: HPC software engineer, GPU programmer, performance engineer
  - **Data and AI**: big-data analyst, ML/AI engineer using HPC clusters
  - **National labs and Supercomputing centres**: system architect, HPC support scientist, application specialist.

WHY STUDY HPC : SINGLE THREAD SPEEDUP IS DEAD – MUST EXPLOIT PARALLELISM



Hennessy and Patterson, Turing Lecture 2018, overlaid over "42 Years of Processors Data"
https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/;"First Wave" added by Les Wilson, Frank Schirrmeister
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp